Data-Structures & Algorithms

with

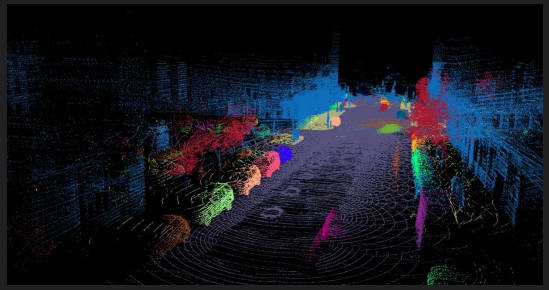
Manoj Prabhakaran

Lecture 31

Union-Find

a.k.a Disjoint Set Union

Partitioning Items into Sets



- A data-set from a LiDAR is a bunch of points in 3D space. Need to identify solid surfaces/objects
  - If two points "touch each other" (i.e., are in adjacent *voxels*) they are part of the same "object"
- Goal: Partition an incoming stream of points into objects
- Operations to support:
  make-set: create a new set with a single element
  find-set: given an element, find the set it is part of
  union: merge two sets into one set

When a point x arrives, make-set(x), and for each of its neighbours y, call union(x,y)

Attendance Break!

Login using CSE LDAP id at

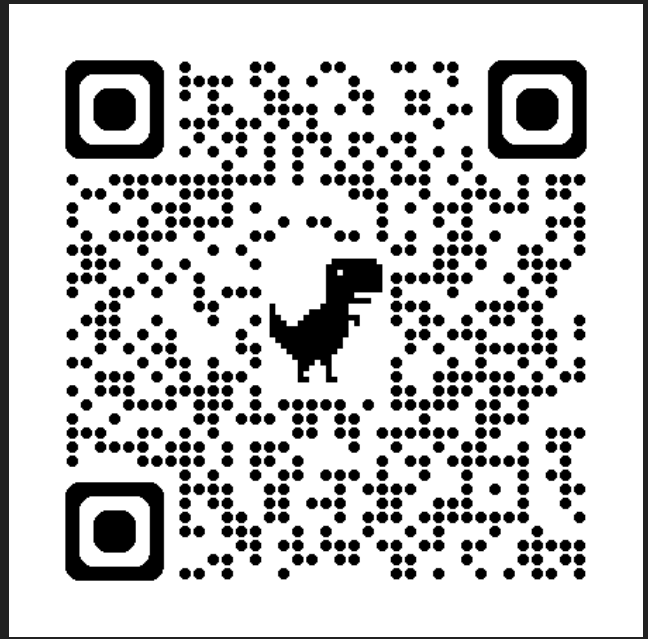
https://www.cse.iitb.ac.in/course_vm/cs213

Read the question, choose the answer and submit.
The points are mainly for attempting,
with a small bonus for getting it right.

The system will select a few random students
who submitted

The selected students should come to the front

If you are not in the classroom, a report will be sent to DADAC!



Partitioning Items into Sets

- A first attempt
  - Keep elements in a standard data structure (say, a hash table). In each element, include a set ID (just an integer)
    - New make-set calls get a new ID: Can maintain a counter of last set ID used, or can use the key for the element itself.
  - make-set and find-set: O(1)
  - union: When taking the union of two sets, replace the set ID of all the elements in one of the sets with that of the other set
    - O(n) time!

A Forest

- Goal: (essentially) O(1) amortised time for make-set, find-set and union
- Challenge:
  - When two sets are combined, cannot afford to update individual elements in either set
- Idea: Each set is organised as a shallow tree (only parent pointers needed) and the item at the root of the tree serves as the set ID
  - A collection of trees is called a forest
  - union takes O(1) time: Merging two trees by simply attaching the root of one as the child of the other
  - make-set is O(1) too: just add a new tree with a single node to the forest
  - But find-set requires traversing parent pointers to the root

Working with Trees

- Each set is organised as a shallow tree (only parent pointers needed) and the item at the root of the tree serves as the set ID
- find-set (after locating the item's node, say via a hash table) requires traversing parent pointers to the root
  - The tree nodes are not limited in arity: so in the best case, all nodes are children of the root
  - In the worst case the tree is a chain of $\Theta(n)$ elements, and find-set takes time $\Theta(n)$
    - Each union operation can attach a large set to a set with one element and grow its height by 1
- Can we reduce find-set time while keeping union $O(1)$?

Union by Size

- Idea: In union, attach the smaller tree under the bigger tree's root
  - Intuition: Dissuades the bigger tree from getting taller
- What is the maximum height a tree with n items can have?
- For item x, let us track its depth, d(x), and the size of the set it is part of, S(x)
  - When x is created by make-set, d(x)=0, S(x)=1
  - In each union operation: If x's root stayed on as the root, then d(x) doesn't change, and S(x) doesn't decrease. Otherwise, d(x) increases by 1 and S(x) at least doubles (because the other set was not smaller)
    - Whenever d(x) increases, S(x) doubles (and S(x) never decreases)
    - So $S(x) \geq 2^{d(x)}$ or equivalently, $d(x) \leq \log_2 S(x)$

Better than O(log n)

- Idea: In union, attach the smaller tree under the bigger tree's root
  - Intuition: Dissuades the bigger tree from getting taller
- What is the maximum height a tree with n items can have?
  - O (log n)
- Can we do even better?
  - Remember, in the best-case the height of every node is at most 1
- Idea: Path compression
  - Once we do find-set(x), we *can* directly attach x to its root
  - Not just x, all the nodes in the path from root to x can be updated to point to the root directly

Path Compression

- Path Compression does not change the worst-case find-set time (the first find-set is the same as without path compression)
- But if there are several find-set queries, the amortised time is much less
- Let us consider the case when a sequence of k find-set queries are made on n items, with no intervening unions
  - Time taken in a find-set = O(#nodes moved to under the root) + O(1)
    - Even if no node is moved, we take O(1) time to find the root
  - Each item can be moved up to under its root at most once
  - So total time for k find-set queries = O(n+k)
- If $k = \Omega(n)$, the amortised time is O(1)

Path Compression

- Amortised time for $\Omega(n)$ find-set queries without intervening unions is $O(1)$
- What if there are intervening unions?
  - A node moved to under its root can again be moved, if that root got connected to the root of another tree due to a union
  - Cannot bound the total number of moves by O(n), or the amortised cost by O(1), any more
  - Turns out, it is practically O(1) still
    - m operations, involving n make-sets, takes overall time $O(m\ \alpha(n))$, where α is the inverse Ackermann function
    - For all practical n, $\alpha(n) \leq 4$

Representation of Nodes

- In union-find, the elements are never deleted
- Allows representing all the nodes in the forest as a vector `A`
  - Once a node is added to `A`, its index in `A` never changes
  - Can use the index in `A` instead of a pointer
- For each node we should store: if it is not a root node, its parent's index, and if it is a root node, the size of its tree (for union by size)
  - A trick: `A[i]` is a signed integer s.t. if `A[i] ≥ 0`, then `parent(i) = A[i]` and if `A[i] < 0`, then node i is a root of a tree with size `-A[i]`
  - The forest is just `std::vector<int>`

An Extension

- Suppose we would also like to support enumeration of a set (returned by find-set) in worst-case time proportional to the size of that set (independent of n)
  - Our trees only carry parent pointers: cannot traverse the tree from the root
  - Option 1: Maintain child pointers and traverse the tree from the root
    - Since arity is unbounded, maintain children as a linked list at each node. While flattening, move all the siblings of each ancestor also directly under the root.
  - Option 2 (simpler): Maintain a circular linked list (unrelated to the tree's topology) tying all the nodes in a set together
    - Root appears somewhere in this ring, from where all elements can be discovered. During union, merging the rings of two sets takes O(1) time. The rings are not changed during find-set operations.